

Meta-Complexity Notes

UNDERGRADUATES MODERN TCS LEARNING PATH

Qishan Chen
Department of Mathematics
Sun Yat-Sen University

Contents

Introduction to meta complexity Part 1.	2
Introduction to meta complexity Part 2. 10 open problems.	7
Problem 1. NP-Hardness of MCSP	7
Problem 2. Learning AC^0 Circuits	7
Problem 3. A Unifying Hardness Hypothesis Based on Meta-Complexity	8
Problem 4. Ruling Out Heuristica	9
Problem 5. Ruling Out Pessiland	10
Problem 6. Karp Theorem for OWFs	11
Problem 7. Basing Other Cryptographic Primitive on Meta-Complexity	12

Introduction to meta complexity Part 1.

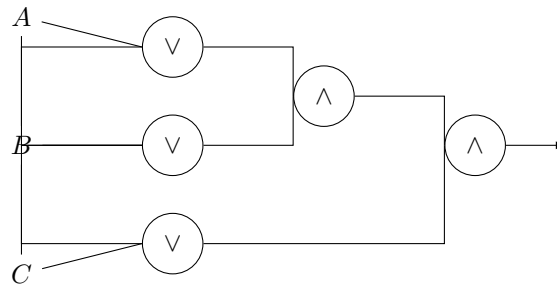
MCSP definition:

The truth table can be like

000 | 001 | 010 | 011 | 100 | 101 | 110 | 111,

of size 2^n , where the input length is 3 in this example. The input is listed in binary order.

Input	000	001	010	011	100	101	110	111
Output	0	0	0	1	0	1	1	1



Circuit example: use and, or, not gate; if no “no”, call this a “monotone circuit” which represent a monotone Boolean function.

$$(A \vee B) \wedge (A \vee C) \wedge (B \vee C).$$

Can compute with **5 gates** (where however truth table have 8 length).

Circuit complexity of a string:

For a circuit C , $\text{tt}(C)$ is the truth table of C , and $\text{cc}(C)$ is the number of gates at C .

For a binary **string** S of length 2^n , the circuit complexity of S , called $\text{cc}(S)$, is the **size of the smallest circuit** which has truth table S . That is:

$$\text{cc}(S) = \min\{\text{size}(C) : \text{tt}(C) = S\}.$$

MCSP is in NP:

MCSP is given a truth table S and a threshold s ; the question is whether $\text{cc}(S) \leq s$.

Given the truth table y (of size $N = 2^n$) of a Boolean function F and parameter s , we guess a circuit C for F of size s , and check that y is the truth table of the function F computed by C , on each input z of size $\log(N)$, and verify if $C(z)$ is consistent with y . Verifying time is $O(N \log N) = \text{poly}(N)$.

black-box problem

↓
MCSP

white-box problem

↓
SAT

Part of the project: **MCSP vs SAT**.

Variant of MCSP for other circuit class:

Given any circuit class $\mathcal{C}$, define the problem $\mathcal{C}$ -MCSP, where the question is whether F has $\mathcal{C}$ -circuits of size s (F is a Boolean function and s is a parameter).

Note: $\mathcal{C}$ more powerful class, $\mathcal{C}$ -MCSP may not be harder! E.g. DNF-MCSP is NP-complete, but MCSP question it!

Other complexity measures:

- K : Kolmogorov complexity.
- Kt : Levin complexity.
- K_s : space-bounded Kolmogorov complexity.
- KT : Allender complexity.
- K^{poly} : poly-time bounded Kolmogorov complexity.

K:

- Let U be a fixed universal Turing machine, ε be the empty string (input), and p denote a program.
- For any string $x \in \Sigma^*$,

$$K(x) = \min\{|p| : U(p, \varepsilon) = x\}.$$

While **conditional Kolmogorov complexity** is: given $y \in \Sigma^*$, for any string $x \in \Sigma^*$,

$$K(x \mid y) = \min\{|p| : U(p, y) = x\}.$$

Intuitively, $K(x)$ is the size of the smallest program that produces x when we run it on the empty string.

- Simple examples:

1. $K(0^n) \leq \log(n) + O(1)$, because describe “ N ” using $\log(n)$ bits in binary computer science.
2. $K(\pi^N) \leq \log(N) + O(1)$, cause deterministic algorithm (like Taylor expansions) exist and first N number of π cost $\log N$ bits.

Basic properties of K :

- (1) For every $x \in \Sigma^*$, $K(x) \leq |x| + O(1)$.
- (2) For each integer n , there exist x of length n such that $K(x) \geq n$. Here $|x| = n$.
 - For any i , there most 2^i strings have Kolmogorov complexity i . Use pigeonhole principle. Exist an x of length n with $K(x) \geq n$.

Meta Complexity of K : MKP

- MKP: input string x together with parameter s . Question: whether $K(x) \leq s$.
- K : given x , compute the K -complexity of x .

MKP and K reduce each other in **poly** time.

Uncomputability of Kolmogorov complexity.

That is, no TM M can compute all the string's Kolmogorov complexity.

Proof. Suppose for a sake of contradiction there is a TM M that computes K . Define a TM N . For any input x , N first uses M to compute $K(x)$, and accepts x iff $K(x) \geq n$.

By property (2), N accepts at least one string for each input length n .

Now define a sequence of strings $\{x_n\}$, and $|x_n| = n$:

- (a) For each n , x_n is the lexicographically first string of length n that N accepts.
- (b) Note that we compute x_n by simulating N on strings of length n in lex order and outputting the first string N accepts.
- (c) This implies that $K(x_n) \leq \log(n) + O(1)$, which leads contradiction.

- Unavailability. Thm [Cai '74]. Let X be any “effectively axiomatizable sound proof system”. Then only finitely many m for each statement form “ $K(X) \geq m$ ” that can be proved in X .

Proof as same as forward.

Kt .

- Let U be a fixed time-efficient universal TM.

$$Kt(x) = \min\{|p| + \log t : U(p, \varepsilon) = x \text{ in at most } t \text{ steps}\}.$$

- Clearly for each x , $Kt(x) \leq |x| + \log(|x|)$, and for each n , there is an x of $|x| = n$ that $Kt(x) \geq n$.
- Note that $K(x) \leq Kt(x)$ for each x .
- Why plus $\log t$? $p \rightarrow p + 1$ bits, $2^p \rightarrow 2^{p+1}$ searching space, time double $\rightarrow \log$ will only cause it “add 1”.

Meta Complexity of Kt :

MKTP and Kt define as normal as imagine.

- Observation: MKTP and Kt are in EXP time (by doing a brute force search). Fact: MKTP and Kt are in EXP-hard under condition.
- Still, MKTP and Kt reduce each other in poly time.

K_s :

U be fixed space-efficient Universal TM.

$$K_s(x) = \min\{|p| + s : U(p, \varepsilon) = x \text{ using space at most } s\}.$$

Thus:

- (a) $K_s(x) \leq |x| + \log(|x|)$ and for each n , there is an x of length n that $K_s(x) \geq n$.
- (b) $K(x) \leq K_s(x)$.
- Meta complexity of K_s . (MKSP and K_s both in EXP time.)

K^{poly} :

Let t be a fixed poly and U a fixed time-efficient Universal TM.

$$K^t(x) = \min\{|p| : U(p, \varepsilon) = x \text{ in at most } t(|x|) \text{ steps}\}.$$

- (a) $K^t(x) \leq |x| + O(1)$.
- (b) $K(x) \leq K^t(x)$ for each x .
- (c) Exist an x that $K^t(x) \geq n$ for each n .
- Meta complexity of K^{poly} .

MINKT: string x together with input s and t in unary. Question: whether $K^t(x) \leq s$.

MKTP, MINKT, K^t are NP. Open: NP-hard? ($\Leftrightarrow$ one-way function exist. One-way function is harder than P vs NP?)

KT .

- U be fixed time-efficient TM with random access to its tapes.

$$KT(x) = \min\{|p| + t : U(p, i) = x_i \text{ in at most } t \text{ steps for each } i \in [1, |x|]\}.$$

Reading index i takes $\log i$ bits, so $KT(x) \leq |x| + \log |x|$; **binary code**.

Proposition Why KT important?

Let y be the truth table with BF F . Then $KT(y)$ and circuit size of F are poly related. (Thus interpret KT as Kolmogorov-theoretic analogue of cc .)

Meta-complexity of KT .

- MKTP in NP. MKTP and KT reduction in poly.

The approximating setting.

- Two nature notions of approximating.
- (a) Approximating of the complexity of a string $\rightarrow$ worst case to average case reduction of meta complexity.
 - (b) Approximate by low complexity string (apply of learning theory).

Meta-complexity as a Tool:**①. Relevance to cc . (kind of tautology.)**

- If MCSP does not have circuits of size $N^{1-o(1)}$, then $P \neq NP$.
- Every No instance (f.s) of MCSP correspond to a circuit lower bound.

②. Relate to learning theory.

- Learning closely related to solving search version of MCSP:
 - (a) Learning model: learner given oracle access to target function and outputs “good” hypothesis (i.e. small circuits) of that target function if hypothesis exist.
 - (b) Search version of MCSP: given truth table output small circuits.

Efficient learner $\Leftrightarrow$ efficient decision procedure of MCSP.

③. Related to proof complexity and meta-math.

- Existence of natural proof equiv to efficient zero-error average-case of MCSP.

Introduction to meta complexity Part 2. 10 open problems.

Problem 1. NP-Hardness of MCSP

- More generally, which meta-complexity problems in NP, e.g. KT , K^{poly} , MINKT, are also NP-hard?

Obstacles.

1. [KC00] shows that if MCSP is NP-hard under [deterministic honest m-reductions], then EXP does NOT have poly-size circuits (i.e. $\text{EXP} \neq \text{P/poly}$).

Honest: existing a poly p that for any x , $|x| \leq p(|f(x)|)$. For example, if $|x| = n$ then $|f(x)|$ could not be $\log n$.

Deterministic m-reduction: two languages A, B . $x \in A \Leftrightarrow f(x) \in B$, with $f(x)$ can be computed by deterministic TM at poly time.

$\text{EXP} \neq \text{P/poly}$: a decision problem in EXP cannot be solved by poly-size circuit. In EXP, dealing with input x needs $2^{\text{poly}(n)}$ time. This is still a conjecture and hypothesis!

A well known Thm: $\text{P} \subseteq \text{P/poly}$ can be proved by drawing a $O(n^k) \times O(n^k)$ tableau with “Time” $\times$ “Space”.

2. Several follow-up works:

[MW17] show that MCSP is not NP-hard under deterministic polylog-time projections.

- Progress towards NP-hardness:
- [MV18]: DNF-MCSP is NP-hard.
- [HZ22]: Partial MCSP is NP-hard.

} ... (and a lot of progress)

Now leave: MCSP is NP-hard?

Problem 2. Learning AC^0 Circuits

- AC^0 circuits: with AND, OR, NOT gates. $AC^0[m]$ circuits: with AC^0 and MOD_m gates. AC^0 circuits $= \bigcup_{m=1}^{\infty} AC^0[m]$ circuits, allow to use different mod gates (with different m).

- Def of MOD gates: Given k **fan-in** $x_1, x_2, \dots, x_k \in \{0, 1\}$, and a definite integer m .

$$\text{MOD}_m(x_1, \dots, x_k) = \begin{cases} 0, & \text{if } \sum_i x_i \equiv 0 \pmod{m}, \\ 1, & \text{if } \sum_i x_i \not\equiv 0 \pmod{m}. \end{cases} \quad (\text{OR converse!}).$$

E.g. MOD_2 gates = XOR gates.

- Is there a quasi-poly time algorithm to learn AC^0 circuits over the uniform distribution with membership queries?
- Explanation: quasi-poly means $2^{\text{polylog}(n)}$ etc.

Learning Circuits. Consider the “black box circuits” as a “unknown function”. Algorithm guess what the “black box circuits” like through input and output.

Membership Queries. The learning algorithm can ask the oracle (black box) “subjectively” and get output $f(x)$ on time.

Uniform distribution. The input x is uniformly selected.

- Breakthrough: [CIKK16] show that such a learning algorithm exist for $AC^0[p]$ circuits for any prime p , but still open for AC^0 . That is, which class of circuits are learnable?
- Connection with Meta-Complexity. (nature proofs).

[CIKK16] shows: natural proofs of $\mathcal{C} \rightarrow$ learning algorithms for $\mathcal{C}$.

Natural proofs useful against $\mathcal{C}$ is essentially an efficient zero-error average-case algorithm for $\mathcal{C}$ -MCSP.

- Reverse the connection: Lower Bounds for learning?

If we could derive non-trivial learning algorithm for $\mathcal{C} = AC^0$ or $\mathcal{C} = \text{depth-2 } TC^0$, we could get the new lower bound that BPEXP not in $\mathcal{C}$.

(bounded-error probabilistic EXP)

Problem 3. A Unifying Hardness Hypothesis Based on Meta-Complexity

- Hardness hypothesis example: $NP \neq P$, existence of OWF's, ETH, SETH. But these hardness hypothesis seems uncorrelated.

Problem: Is there a hardness hypothesis based on meta-complexity that can be used to **unify** some of these results?

E.g. (a possible hardness hypothesis): inapproximability of circuit size up to $N^{1-\varepsilon}$ factor.

Algorithmica ($P = NP$)	Heuristica (solve problem on average)	Pessimism	Minicrypt	Cryptomania (have crypto)
NP is worst-case hard NP is average-case hard one-way function exist public key crypto exist				

Problem 4. Ruling Out Heuristica

- Problem: Does worst-case hardness of NP imply average-case hardness of NP?
- Various versions: worst-case of NP hardness could be respect to $P \mid BPP \mid ZPP \mid \text{size}(\text{poly})$, average-case hardness could be zero-error or bounded-error, etc. We don't know any of these self-reduction.

For example: worst-case of hardness NP vs BPP: existing problem in NP that cannot be reduced to BPP ($NP \not\subseteq BPP$).

- Obstacles to progress.

In some cases, black-box non-adaptive worst-case to average-case reductions for NP imply the collapse of the polynomial hierarchy.

polynomial Hierarchy: Introduction.

- co-NP: For every “no-instance” we have a poly-length certificate and a poly-time algorithm that can be used to verify.
- Oracle definition of polynomial hierarchy:

Define

$$\Delta_0^P := \Sigma_0^P = \Pi_0^P := P.$$

Then for any $i \geq 0$, define

$$\Delta_{i+1}^P := P^{\Sigma_i^P}, \quad \Sigma_{i+1}^P := NP^{\Sigma_i^P}, \quad \Pi_{i+1}^P := coNP^{\Sigma_i^P}.$$

Here P^A is the set of decision problems solvable in poly time by a TM augmented by an oracle for some complete problem in class A .

The classes NP^A and $coNP^A$ are defined analogously.

For eg. $\Sigma_1^P = NP^P = NP$ and $\Pi_1^P = coNP^P = coNP$, but $\Delta_2^P = P^{NP}$ (much stronger than $P \mid NP \mid coNP$).

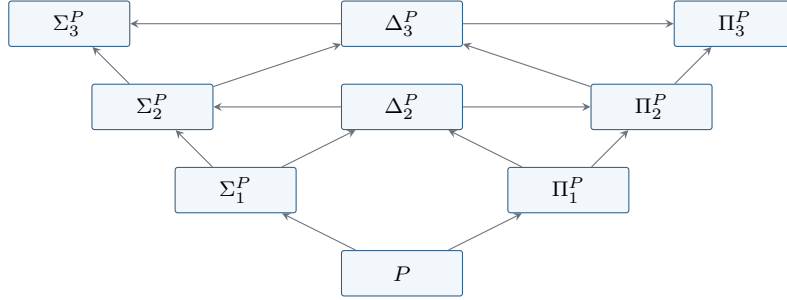
- Relations in the polynomial hierarchy:

The union of all classes in the polynomial hierarchy is called the complexity class PH .

The definitions imply the relations:

$$\begin{aligned}\Sigma_i^P &\subseteq \Delta_{i+1}^P \subseteq \Sigma_{i+1}^P, \\ \Pi_i^P &\subseteq \Delta_{i+1}^P \subseteq \Pi_{i+1}^P, \quad \text{can be verified easily.} \\ \overline{\Sigma}_i^P &= \Pi_i^P.\end{aligned}$$

We can express this as a communicative diagram:



The arrows denotes inclusion.

Question: Whether any of these inclusions are **proper**?

“Collapse”: If any $\Sigma_k^P = \Sigma_{k+1}^P$, or $\Sigma_k^P = \Pi_k^P$, then the hierarchy collapse to level k : for all $i \geq k$, $\Sigma_i^P = \Sigma_k^P$.

In particular:

- (1) $NP = P$ iff $P = PH$ (hierarchy collapse to Σ_0^P). Then also $NP = coNP$.
- (2) $NP = coNP$ then $NP = PH$ (i.e. hierarchy collapse to Σ_1^P).

- Another def of Σ and Π : $y_1 - \dots - y_k$ poly-certif. R poly verifier.

$$\begin{aligned}\Sigma_k^P : \quad & x \in L \Leftrightarrow \exists y_1 \forall y_2 \exists y_3 \dots Q_k y_k : R(x, y_1, y_2, \dots, y_k) = 1 \quad (\text{begin with } \exists), \\ \Pi_k^P : \quad & x \in L \Leftrightarrow \forall y_1 \exists y_2 \forall y_3 \dots Q'_k y_k : R(x, y_1, \dots, y_k) = 1 \quad (\text{begin with } \forall).\end{aligned}$$

Problem 5. Ruling Out Pessiland

- Does average-case hardness of NP imply that OWFs exist?
- Note that average-case hardness of specific problems is known to imply OWFs (e.g. factoring or discrete log). The question is whether general average-case hardness implies?
- [OWB] give a relativized world in which Pessiland exist. That means any proof technique used here cannot be relativize.

The relativization barrier.

- The relativization barrier.

A proof relativizes if it remains valid when every machine is given access to the same arbitrary oracle A . In fact, the proof treats the internal structure of this computation as a black box and uses only general properties of oracle access.

The BGS Thm shows:

$$1^\circ. \exists A, P^A = NP^A, \quad 2^\circ. \exists B, P^B \neq NP^B.$$

That is, relativized proof could not be used to proof P vs NP .

- Progress via Meta-Complexity.

[LP20]: showed OWFs exist if K^{poly} is hard on average over the uniform distribution (to the distribution of input x).

[LP22]: showed that conditional K^{poly} (that is, $K^t(x \mid y)$) is worst-case NP-hard, and that the hardness of average case of conditional K^{poly} over uniform distribution implies that OWFs exists.

Problem 6. Karp Theorem for OWFs

- Problem: Are there variety of natural distribution problems whose average-case hardness is equivalent to the existence of OWFs?
- Analogy to NP-completeness:

Cook and Levin shows that SAT is NP-hard.

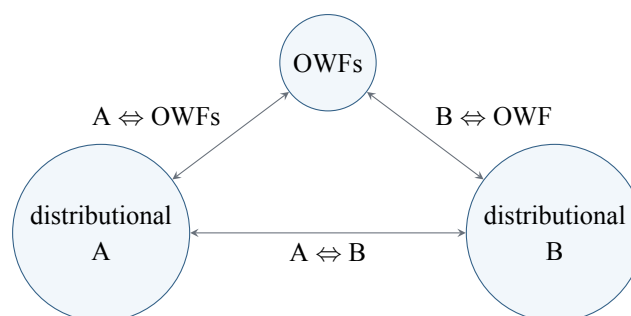
$\Downarrow$

Karp show that 21 other problems are also NP-hard by reduction to 3-SAT.

- The first natural distributional problem that is OWF-complete was given by [LP20].

Further results given by [LP22, IL022], but all of these cases are meta-complexity problems. So question is:

- Are there some natural **other** distributional problems that is not a meta-complexity flavour, equal to these results? Questions above?
- Motivation: It is notoriously hard to reduce natural distributional problems to each other. E.g. reduction between 3-SAT and 4-SAT over uniform distribution unknown.
- One path towards achieving these reductions is using OWFs as a “middle station”. This path is used to proof equivalence of different meta-complexity problems distributional. Diagrams below show how:



To show $A \Leftrightarrow B$, we just need to show $A \Leftrightarrow \text{OWFs}$ and $B \Leftrightarrow \text{OWFs}$.

Problem 7. Basing Other Cryptographic Primitive on Meta-Complexity

- Problem: Are there natural meta-complexity assumptions that yield other cryptographic primitive, including trapdoor functions, collision-resistant hashing functions, or indistinguishability obfuscation?
- Fact: We have been successful in characterizing OWFs in multi-way.